A7 Observability

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track A: Databricks pipeline engineering 2
A7 Observability 3
A7-L1 Structured logging with the logging module, not print 3
A7-L2 A pipeline run log table with an explicit schema 7
A7-L3 Alert on failures, and on the silence that hides them 13
Track A best-practice checklist 18
Appendix: solutions and answer keys 19

Track A: Databricks pipeline engineering

Build Databricks pipelines that are correct, fast, observable and tested.

A7 Observability

Roadmap: Months 16 to 18: MLOps and deployment; Current work. Structured logging and pipeline run log tables.

A7-L1 Structured logging with the logging module, not print

Concept

`print` writes text to standard output, with no level, no source and no context. In a scheduled job, that text lands in a driver log that nobody searches until something breaks.

Python's standard `logging` module fixes all three problems.

One logger per module. `log = logging.getLogger(__name__)` names the logger after the module, such as `pipelines.silver`. Every line then says where it came from.

Levels. `DEBUG` for detail while developing, `INFO` for normal milestones, `WARNING` for something odd that did not stop the run, `ERROR` for a failure, `CRITICAL` for a failure that stops everything. Loggers pass `WARNING` and above by default, so `log.info(...)` prints nothing until you configure a level.

Context on every line. Include the identifiers you will search for later: `run_id`, `tracker_id`, `wave_id`, `layer`. A `LoggerAdapter` attaches them once, and a format string prints them:

```
1 base = logging.getLogger(__name__)
2 log = logging.LoggerAdapter(base, {"run_id": run_id, "wave_id": wave_id})
3 log.info("merged rows=%d", n)
4 # INFO pipelines.silver run_id=7f3c wave_id=W14 merged rows=1832
```

Three more rules:

- Pass values as arguments, `log.info("rows=%d", n)`, not pre-built f-strings. The logger formats them only when the line is actually emitted.
- Inside `except`, call `log.exception("...")`. It logs at ERROR and appends the full traceback.
- Configure handlers and formats once, in the entry point, never inside library functions.

Logs are for people reading one run. Metrics you want to compare across runs, such as row counts and statuses, also go to the run log table (lesson A7-L2).

Why it matters

A silver job prints "done" at the end of each step. One night it finishes "successfully" with zero rows merged, because an upstream filter used the wrong wave ID. The driver log for that run holds 400 lines of "done" and Spark noise, with no wave ID and no row counts.

With `log.info("merged rows=%d", n)` and the wave ID on every line, the zero is visible in one search, and a WARNING for an empty merge would have flagged it at 02:00.

Worked example: weak code

```
1  def merge_wave(df, wave_id):
2      print("starting")
3      try:
4          n = do_merge(df)
5          print(f"done {n}")
6      except Exception as e:
7          print("error:", e)
```

1. **Line 2.** No level, no module, no wave ID: unsearchable.
2. **Line 5.** A count with no label. Zero rows looks the same as a normal run.
3. **Line 7.** Swallows the exception and loses the traceback. The job reports success.

Worked example: strong code

```
1  import logging
2
3  logger = logging.getLogger(__name__)
4
5
6  def merge_wave(df: DataFrame, run_id: str, wave_id: str) -> int:
7      """MERGE one wave, logging with run and wave context. Re-raises on failure."""
8      log = logging.LoggerAdapter(logger, {"run_id": run_id, "wave_id": wave_id})
9      log.info("merge started")
10     try:
11         rows = do_merge(df)
12     except Exception:
13         log.exception("merge failed")
14         raise
15     if rows == 0:
16         log.warning("merge wrote 0 rows")
```

```
    else:
        log.info("merge finished rows=%d", rows)
    return rows
```

1. **Line 3.** One logger per module, named after it.
2. **Line 8.** Context attached once, printed on every line by the entry point's format.
3. **Line 13.** ERROR level plus the full traceback.
4. **Line 14.** Re-raise, so the job fails and alerts fire.
5. **Line 16.** A suspicious result gets its own level, not a buried number.

Every line says what, where, which run and which wave. Failures keep their traceback and still fail the job.

Recap card

- **Create a module logger** logger = logging.getLogger(**name**)
- **Default level threshold with no configuration** WARNING. log.info(...) prints nothing until the entry point sets a level.
- **Attach run_id and wave_id to every line** logging.LoggerAdapter(logger, {"run_id": ..., "wave_id": ...})
- **Inside except** log.exception("...") for ERROR plus traceback, then a bare raise.

Practice

Exercise 1. Predict the output (level 1)

No logging configuration has run. What appears in the output?

```
import logging

log = logging.getLogger("silver")
log.info("loaded")
log.warning("3 unexpected labels")
```

Exercise 2. Fill the gap (level 2)

Create a module logger, then log the failure with its traceback and re-raise.

```
logger = logging.getLogger(_____(1))

try:
    rows = do_merge(df)
except Exception:
    logger._____(2)("merge failed")
    _____(3)
```

Exercise 3. Spot the bug (level 3)

This job failed at the MERGE last night, yet the scheduler marked it as succeeded and nobody was alerted. Tap the line, then fix the function.

```
1 def merge_wave(df, run_id: str, wave_id: str) -> int:
2     """MERGE one wave. Logs failures."""
3     log = logging.LoggerAdapter(logger, {"run_id": run_id, "wave_id":
    wave_id})
4     try:
5         return do_merge(df)
6     except Exception:
7         log.exception("merge failed")
8         return 0
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Write the function (level 4)

Write `wave_logger(run_id, wave_id)` that returns a `logging.LoggerAdapter` around the module logger `logging.getLogger("pipelines.silver")`, carrying `run_id` and `wave_id`. Then write `log_merge_result(log, rows)`: log at WARNING `"merge wrote 0 rows"` when rows is 0, otherwise at INFO `"merge finished rows=%d"` with the count passed as an argument.

Your code must pass these checks:

- The INFO line must carry run_id and wave_id and read `merge finished rows=5`.
- Zero rows must log at WARNING: `merge wrote 0 rows`.
- Wrap the logger named `pipelines.silver`.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
1  # Module logger with run and wave context on every line.
2  def wave_logger(
3      run_id: str,
4      wave_id: str,
5  ) -> logging.LoggerAdapter: ...
6
7  # INFO for a normal merge, WARNING for an empty one.
8  def log_merge_result(
9      log: logging.LoggerAdapter,
10     rows: int,
11 ) -> None: ...
```

Starter code:

```
1 import logging
2
3
```

```python
def wave_logger(run_id: str, wave_id: str) -> logging.LoggerAdapter:
    ...


def log_merge_result(log: logging.LoggerAdapter, rows: int) -> None:
    ...
```

Exercise 5. Architecture design (level 5)

Choose the level for each event in a silver job.

Options: DEBUG, INFO, WARNING, ERROR.

1. Run started for wave W14
2. MERGE wrote 0 rows for a wave that should have data
3. MERGE raised DELTA_MULTIPLE_SOURCE_ROW_MATCHING_TARGET_ROW_IN_MERGE
4. The full query plan of the MERGE source, for troubleshooting

Sources

- Without configuration, loggers pass WARNING and above, and the last-resort handler prints only the message; LoggerAdapter adds context; logger.exception includes the traceback. (verified: Run in Python 3.11 in this repository.)

A7-L2 A pipeline run log table with an explicit schema

Concept

Logs describe one run. A **run log table** lets you compare every run: which waves loaded, how many rows, how long, and which failed. It is a Delta table, append-only, with one row per pipeline run per layer.

A useful schema:

Column	Type	Notes
`run_id`	STRING, not null	`str(uuid.uuid4())`, shared by every log line of the run
`pipeline_name`	STRING, not null	`nat_silver_pipeline`
`tracker_id`, `wave_id`, `layer`	STRING	what the run touched
`rows_processed`	BIGINT, not null	from Delta commit metrics
`status`	STRING, not null	`SUCCESS` or `FAILED`

Column	Type	Notes
`error_message`	STRING, nullable	NULL on success
`started_at`, `finished_at`	TIMESTAMP, not null	UTC

Build the record against an **explicit** `StructType`. Your reference `log_pipeline_run` builds a DataFrame from a dict and lets Spark infer types. That fails on every successful run: `error_message` is `None`, and Spark raises `CANNOT_DETERMINE_TYPE` because it cannot infer a type from a column of only nulls. The explicit schema also rejects a wrong type, such as `"12"` for `rows_processed`, before anything is written.

Take timestamps with `datetime.now(timezone.utc)`. `datetime.utcnow()` is deprecated from Python 3.12 and returns a naive datetime that carries no timezone.

Import everything you use. The reference function calls `uuid` and `datetime` without importing them, a `NameError` on its first run. `ruff check` (rule F821) catches it in CI.

Write the log record in a `finally` block, or in both the success and failure paths, so failed runs are logged too. A run log that only records successes cannot tell you how often the pipeline fails.

Why it matters

The reference `log_pipeline_run` has never succeeded. It fails first with `NameError: name 'uuid' is not defined`. Once someone adds the imports, it fails on every successful run with `CANNOT_DETERMINE_TYPE`, because `error_message=None` is the only value in its column. The orchestrator catches that error "so logging never breaks the pipeline". Months later the team wants to show a client every load of 2025. The run log table is empty.

An explicit schema, imported names and a test that writes one record on a local SparkSession prevent all three problems.

Worked example: weak code

```
1 def log_pipeline_run(pipeline_name, tracker_id, wave_id, layer,
      rows_processed, status, error_message: str = None):
2     run_record = spark.createDataFrame([{
3         "run_id": str(uuid.uuid4()),
4         "pipeline_name": pipeline_name,
5         "rows_processed": rows_processed,
6         "status": status,
```

```
        "error_message": error_message,
        "run_timestamp": datetime.utcnow().isoformat(),
    }])

run_record.write.format("delta").mode("append").saveAsTable("research_governance.
```

1. **Line 1.** Claims a string, defaults to None (defect 7).
2. **Line 2.** Types inferred from one dict. An all-None column raises CANNOT_DETERMINE_TYPE.
3. **Line 3.** uuid is never imported: NameError (defect 1).
4. **Line 8.** datetime not imported, utcnow deprecated (defect 6), and the timestamp stored as a string.

Worked example: strong code

```
import uuid
from datetime import datetime, timezone

from pyspark.sql import DataFrame, SparkSession
from pyspark.sql import types as T

RUN_LOG_SCHEMA = T.StructType([
    T.StructField("run_id", T.StringType(), False),
    T.StructField("pipeline_name", T.StringType(), False),
    T.StructField("wave_id", T.StringType(), True),
    T.StructField("rows_processed", T.LongType(), False),
    T.StructField("status", T.StringType(), False),
    T.StructField("error_message", T.StringType(), True),
    T.StructField("started_at", T.TimestampType(), False),
    T.StructField("finished_at", T.TimestampType(), False),
])


def new_run_id() -> str:
    """A unique id shared by the log lines and the run log row of one run."""
    return str(uuid.uuid4())


def run_record(
    spark: SparkSession,
    run_id: str,
    pipeline_name: str,
    wave_id: str | None,
    rows_processed: int,
    status: str,
    started_at: datetime,
    error_message: str | None = None,
) -> DataFrame:
    """One run log row, typed by RUN_LOG_SCHEMA."""
    finished_at = datetime.now(timezone.utc)
```

```
36      row = (run_id, pipeline_name, wave_id, rows_processed, status,
        error_message, started_at, finished_at)
4       return spark.createDataFrame([row], RUN_LOG_SCHEMA)
```

1. **Line 7.** The schema is declared once. Nothing is inferred.
2. **Line 13.** Nullable string: None is fine, because the type is declared.
3. **Line 35.** Timezone-aware UTC, stored as a real TIMESTAMP.
4. **Line 37.** A wrong type, such as "12" for rows_processed, fails here with a clear message.

The strong record builds on every run, success or failure, and its types never depend on the values in one row.

Recap card

- **Why does the reference log_pipeline_run fail on success?** error_message=None is inferred with no type: CANNOT_DETERMINE_TYPE. Declare a StructType.
- **UTC timestamp in modern Python** datetime.now(timezone.utc). utcnow() is deprecated from 3.12 and naive.
- **Which ruff rule catches the missing uuid import?** F821, undefined name.
- **Where do you write the run record so failures are logged too?** In a finally block, after setting status in try and except.

Practice

Exercise 1. Multiple choice (level 1)

`spark.createDataFrame([{"run_id": "r1", "error_message": None}])` runs on PySpark 4.0. What happens?

- a) error_message becomes a STRING column holding NULL.
- b) It fails with CANNOT_DETERMINE_TYPE.
- c) error_message is dropped from the DataFrame.
- d) error_message becomes a NULL-typed column that Delta accepts.

Why?

- r1) Spark infers each column's type from the data, and a column holding only None gives it nothing to infer from.
- r2) Spark never allows None in a DataFrame.

Exercise 2. Fill the gap (level 2)

Complete the imports and the timestamp so the record code runs on Python 3.12 without deprecation warnings.

```
import _____(1)
from datetime import datetime, _____(2)

run_id = str(uuid.uuid4())
started_at = datetime._____(3)(timezone.utc)
```

Exercise 3. Spot the bug (level 3)

This builds a run log row. It fails on every successful run. Tap the line, then fix the function so it declares its schema.

```
def run_record(spark: SparkSession, run_id: str, rows: int, status: str,
    error_message: str | None) -> DataFrame:
    """One run log row."""
    return spark.createDataFrame([{
        "run_id": run_id,
        "rows_processed": rows,
        "status": status,
        "error_message": error_message,
    }])
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Write the function (level 4)

Write `run_record(spark, run_id, pipeline_name, rows_processed, status, started_at, error_message=None)` returning a one-row DataFrame typed by a module constant `RUN_LOG_SCHEMA` with columns `run_id`, `pipeline_name`, `rows_processed` (LongType), `status`, `error_message` (nullable), `started_at` and `finished_at` (TimestampType). Set `finished_at` with `datetime.now(timezone.utc)`.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Module constant.
RUN_LOG_SCHEMA = ...

# One run log row typed by RUN_LOG_SCHEMA.
def run_record(
    spark: SparkSession,
    run_id: str,
    pipeline_name: str,
    rows_processed: int,
    status: str,
    started_at: datetime,
    error_message: str | None = None,
```

```
) -> DataFrame: ...
```

Starter code:

```python
from datetime import datetime, timezone

from pyspark.sql import DataFrame, SparkSession
from pyspark.sql import types as T

RUN_LOG_SCHEMA = ...


def run_record(spark: SparkSession, run_id: str, pipeline_name: str,
    rows_processed: int, status: str, started_at: datetime, error_message:
    str | None = None) -> DataFrame:
    ...
```

Exercise 5. Put in order (level 5)

Order the orchestrator's steps so both successful and failed runs are logged, and a failure still fails the job.

- except: set status FAILED and error_message, log.exception, then raise
- Create run_id and started_at
- finally: append the run record to the run log table
- Set status SUCCESS
- try: run the layer and read rows_processed from the commit metrics

Practical: Write run log records for a successful and a failed run

Platform: Databricks Free Edition. Run this in Databricks Free Edition. It creates a run log table with an explicit schema, runs one step that succeeds and one that fails, and records both. It also shows the inference error the reference code hits.

Download the notebook `A7-L2.ipynb` from the app.

Sources

- createDataFrame from a dict with an all-None column raises CANNOT_DETERMINE_TYPE; an explicit StructType accepts None and rejects '12' for LongType with FIELD_DATA_TYPE_UNACCEPTABLE_WITH_NAME. (verified: Run on PySpark 4.0.4 in this repository.)
- datetime.utcnow() is deprecated from Python 3.12. (verified: Python 3.12 release notes (What's New in Python 3.12, deprecated APIs); CLAUDE.md standard.)

A7-L3 Alert on failures, and on the silence that hides them

Concept

An alert is useful when it fires for real problems, reaches the person who can act, and says enough to start. Pipelines need two kinds.

1. Loud failures: the job fails. Databricks job notifications send an email or a webhook message when a run fails. In a job definition or bundle:

```
email_notifications:
  on_failure:
    - data-oncall@example.org
```

These fire only if the job actually fails. Every `except` block that logs and carries on silences them. Log, record the run as FAILED, then re-raise (lesson A7-L2).

Job-level notifications are not sent when a failed task is retried. If you want a message for every failed attempt, add task-level notifications.

2. Silent failures: the job succeeds, but the data is wrong or missing. A job that never starts, a MERGE that writes 0 rows, a wave with a WARN on every contract check. No job fails, so no notification fires. Detect these with queries on your run log and QA results tables, run on a schedule, for example as Databricks SQL alerts:

- No SUCCESS run for a pipeline in the last 26 hours (a daily job plus slack).
- A run with `rows_processed = 0`.
- More WARN results for a wave than the last five waves' average.

Write each alert query against the exact values your tables hold. A query for `status = 'FAIL'` never fires on a table that stores `FAILED`.

Send alerts to a group address or channel, never to one person's inbox. Include the pipeline, the wave, the run ID and a link, so the reader can start without searching.

Why it matters

A nightly bronze job's schedule is paused during a workspace migration and never resumed. The job does not fail, because it never runs, so no failure email arrives. Silver and gold keep succeeding on the old data. Eleven days later a client asks why the dashboard has not moved.

A scheduled query, "no SUCCESS for nat_bronze_pipeline in 26 hours", would have alerted the on-call group on the second morning.

Worked example: weak code

```
1  -- Alert: failed runs today
2  SELECT COUNT(*) AS failures
3  FROM research_governance.pipeline_run_log
4  WHERE status = 'FAIL'
5    AND run_timestamp >= current_date();
```

1. **Line 2.** Counts failures only. A job that stopped running produces no rows and no alert.
2. **Line 4.** The table stores FAILED. This alert has never fired.

Worked example: strong code

```
1  -- Alert 1: pipelines with no successful run in the last 26 hours.
2  SELECT pipeline_name,
3         max(finished_at) AS last_success
4  FROM research_prod.research_governance.pipeline_run_log
5  WHERE status = 'SUCCESS'
6  GROUP BY pipeline_name
7  HAVING max(finished_at) < current_timestamp() - INTERVAL 26 HOURS;
8
9  -- Alert 2: successful runs that wrote nothing in the last day.
10 SELECT pipeline_name, wave_id, run_id
11 FROM research_prod.research_governance.pipeline_run_log
12 WHERE status = 'SUCCESS'
13   AND rows_processed = 0
14   AND finished_at >= current_timestamp() - INTERVAL 1 DAY;
```

1. **Line 7.** Detects silence: a pipeline whose last success is too old, whether it failed or never ran.
2. **Line 10.** The alert names what to look at: pipeline, wave and run.
3. **Line 13.** A success that did nothing is a silent failure worth an alert.

Job notifications catch runs that fail. Scheduled queries catch runs that do not happen, or that succeed at doing nothing.

Recap card

- **Why must an except block re-raise?** Otherwise the run succeeds and on_failure notifications never fire.
- **Job-level versus task-level failure notifications** Job-level notifications are not sent for retried task failures. Use task-level ones to hear about every failed attempt.
- **How do you detect a pipeline that stopped running?** A scheduled query: no SUCCESS in the run log within the expected window.
- **Where should alerts go?** A group address or channel, with pipeline, wave, run ID and a link.

Practice

Exercise 1. Multiple choice (level 1)

A job has `email_notifications.on_failure` configured. Its silver task catches every exception, logs it, and returns normally. The MERGE fails. What arrives in the on-call inbox?

- a) A failure email.
- b) Nothing.
- c) A warning email.
- d) A success email.

Why?

- r1) The task returns normally, so the run succeeds and on_failure never fires.
- r2) Notifications only work for SQL tasks.

Exercise 2. Fill the gap (level 2)

Complete the job settings so a failed run emails the on-call group.

```
email_notifications:
  _____(1):
    - _____(2)
```

Exercise 3. Spot the bug (level 3)

The run log stores status values `SUCCESS` and `FAILED`. This alert has never fired, even on nights with failures. Tap the line, then fix the query.

```
SELECT pipeline_name, run_id, error_message
FROM research_prod.research_governance.pipeline_run_log
WHERE status = 'FAIL'
  AND finished_at >= current_timestamp() - INTERVAL 1 DAY;
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Write the function (level 4)

Write `stale_pipelines(runs, now, max_age_hours=26)`. `runs` is a list of dicts with `pipeline_name`, `status` and `finished_at` (timezone-aware datetime). Return the sorted names of pipelines whose most recent SUCCESS is older than `max_age_hours` before `now`. A pipeline with runs but no SUCCESS at all is stale too.

What you need to know.

A `timedelta` is a length of time. Subtract one from a `datetime` to get an earlier moment, then compare datetimes with `<` and `>`:

```
from datetime import UTC, datetime, timedelta

now = datetime(2026, 3, 10, 9, 0, tzinfo=UTC)
week_ago = now - timedelta(days=7)        # 2026-03-03 09:00 UTC
datetime(2026, 3, 1, tzinfo=UTC) < week_ago   # True: older than a week
```

`d.setdefault(key, value)` adds `key` with `value` only when the key is missing, and returns what is stored.

Your code must pass these checks:

- A success 5 hours ago is fresh.
- A last success 30 hours ago is stale with the 26-hour limit.
- A pipeline with only FAILED runs has no success at all, so it is stale.
- Use the most recent SUCCESS. A later FAILED does not make a recent success stale.
- Use the max_age_hours argument and return names sorted.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Pipelines whose latest SUCCESS is older than max_age_hours, or that never succeeded.
def stale_pipelines(
    runs: list[dict[str, object]],
    now: datetime,
    max_age_hours: int = 26,
) -> list[str]: ...
```

Starter code:

```
from datetime import datetime, timedelta


def stale_pipelines(runs: list[dict[str, object]], now: datetime, max_age_hours: int = 26) -> list[str]:
    ...
```

Exercise 5. Architecture design (level 5)

Choose how each problem should be detected.

Options: Job failure notification, Scheduled query on the run log or QA tables, No alert needed.

1. The silver MERGE raises an exception
2. The bronze job's schedule was paused and nothing has run for two days
3. Gold succeeds but writes 0 rows for a new wave

4. A run took 9 minutes instead of the usual 8

Sources

- Jobs support email_notifications and webhook_notifications with on_start, on_success and on_failure; job-level notifications are not sent when failed tasks are retried; at most 3 destinations per on_failure. (verified: https://docs.databricks.com/aws/en/jobs/notifications (checked 2026-09-25))
- Databricks SQL alerts run a query on a schedule and notify when a condition is met. (verified: https://docs.databricks.com/aws/en/sql/user/alerts/ (checked 2026-09-25))

Track A best-practice checklist

☐ Every function has type hints, a docstring and one job.

☐ No SparkSession is created at import time. Functions receive spark as a parameter.

☐ Functions are imported as F: `from pyspark.sql import functions as F`.

☐ Table names, column maps and thresholds come from config or registry tables.

☐ Every table states its grain in a comment or table property.

☐ Bronze appends. Silver deduplicates the source, then MERGEs. Gold aggregates by every grain column.

☐ Every run writes one record to the run log table with an explicit schema.

☐ CRITICAL checks raise. WARN checks log and continue.

☐ Every division has a guard for zero and null.

☐ Counts come from one agg pass, not repeated count() calls.

☐ Incremental loads compare source and target on the same keys.

☐ Row filters use a SQL UDF with SET ROW FILTER. Grants go to groups.

☐ Tests run on a local SparkSession in CI before every merge.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

A7-L1 Structured logging with the logging module, not print

A7-L1 exercise 1

Hints. (1) What is the default level threshold? (2) Is INFO above or below WARNING? (3) Only the warning appears, as the bare message.

Answer.

```
3 unexpected labels
```

With no configuration, the effective level is WARNING and Python's last-resort handler prints just the message. Configure a level and format in the entry point to see INFO lines with context.

Common wrong answers.

- `loaded 3 unexpected labels`: Without configuration, loggers pass WARNING and above. The INFO line is dropped.
- `WARNING:silver:3 unexpected labels`: That format comes from `logging.basicConfig()`. Without it, Python's last-resort handler prints the message only.

A7-L1 exercise 2

Hints. (1) The module's own name is a built-in variable. (2) One logger method adds the traceback automatically. (3) A bare statement re-raises the current exception.

Answer.

```
logger = logging.getLogger(__name__)

try:
    rows = do_merge(df)
except Exception:
    logger.exception("merge failed")
    raise
```

`getLogger(__name__)` names the logger after the module. `logger.exception` logs at ERROR with the traceback. A bare `raise` keeps the original exception and fails the job.

Common wrong answers.

- g1 = `"root"`: A hardcoded name loses the module path. `__name__` names the logger after the module automatically.
- g2 = `error`: `error` logs at ERROR but without the traceback, unless you add exc_info=True. `exception` includes it.
- g3 = `pass`: `pass` swallows the failure, so the job reports success. Re-raise with a bare `raise`.

A7-L1 exercise 3

Hints. (1) What does the scheduler see when the function returns normally? (2) Returning 0 looks like an empty but successful merge. (3) Replace `return 0` with `raise`.

Answer. The bug is on line 8.

Fixed version:

```
1 def merge_wave(df, run_id: str, wave_id: str) -> int:
2     """MERGE one wave. Logs failures."""
3     log = logging.LoggerAdapter(logger, {"run_id": run_id, "wave_id":
    wave_id})
4     try:
5         return do_merge(df)
6     except Exception:
7         log.exception("merge failed")
8         raise
```

Returning 0 converts a failure into an apparently successful empty run. Log it, then `raise`, so the job fails and failure notifications fire.

Common wrong answers.

- Line 7: Logging the exception with its traceback is right. The problem is what happens next.
- Line 5: The MERGE call is fine. Look at how its failure is handled.

A7-L1 exercise 4

Hints. (1) `logging.LoggerAdapter(logger, {...})` attaches context. (2) Branch on `rows == 0`. (3) `log.warning("merge wrote 0 rows")` and `log.info("merge finished rows=%d", rows)`.

Answer.

```
1 import logging
2
3
4 def wave_logger(run_id: str, wave_id: str) -> logging.LoggerAdapter:
5     """Module logger with run and wave context on every line."""
```

```
    return logging.LoggerAdapter(logging.getLogger("pipelines.silver"),
{"run_id": run_id, "wave_id": wave_id})


def log_merge_result(log: logging.LoggerAdapter, rows: int) -> None:
    """INFO for a normal merge, WARNING for an empty one."""
    if rows == 0:
        log.warning("merge wrote 0 rows")
    else:
        log.info("merge finished rows=%d", rows)
```

The adapter puts run and wave IDs on every line without repeating them in each call. A separate WARNING for an empty merge makes the suspicious case stand out.

A7-L1 exercise 5

Hints. (1) What would you want to see on a normal night? (2) What should stand out without stopping the run? (3) Bulky detail only when debugging.

Answer.

- Normal milestones: INFO.
- A result that is possible but suspicious: WARNING.
- A failure, logged with log.exception before re-raising: ERROR.
- Bulky diagnostic detail: DEBUG, switched on only when needed.

Levels let one log serve two audiences: INFO and above for the nightly check, DEBUG for the engineer troubleshooting.

Common wrong answers.

- zero = `info`: Zero rows usually means an upstream problem. At INFO it disappears among normal lines.
- fail = `warning`: A failed MERGE stops the run. That is ERROR, with the traceback.
- plan = `info`: A full plan on every run floods the log. Keep it at DEBUG.

A7-L2 A pipeline run log table with an explicit schema

A7-L2 exercise 1

Hints. (1) Where does Spark get the column types here? (2) What type can it infer from None alone? (3) It raises.

Answer. b) It fails with CANNOT_DETERMINE_TYPE. Because: r1) Spark infers each column's type from the data, and a column holding only None gives it nothing to infer from.

With no schema, Spark infers types from values. A column of only `None` has no type to infer, so it raises `CANNOT_DETERMINE_TYPE`. Declare a `StructType`.

Common wrong answers.

- `a`: Only with an explicit schema. Inference sees only None and cannot choose STRING.
- `c`: Spark does not drop columns during inference. It raises.
- `d`: Inference fails before any table is involved.
- `b|r2`: Right outcome, wrong reason. None is fine when the schema is declared. The problem is inferring a type from nothing but None.

A7-L2 exercise 2

Hints. (1) The module that generates unique ids. (2) The class that provides utc. (3) `datetime.now(timezone.utc)`.

Answer.

```
1 import uuid
2 from datetime import datetime, timezone
3
4 run_id = str(uuid.uuid4())
5 started_at = datetime.now(timezone.utc)
```

`import uuid` and `from datetime import datetime, timezone` supply every name. `datetime.now(timezone.utc)` gives a timezone-aware UTC timestamp.

Common wrong answers.

- g3 = `utcnow`: `datetime.utcnow()` is deprecated from Python 3.12 and returns a naive datetime. Use `datetime.now(timezone.utc)`.
- g2 = `date`: You need `timezone`, for `timezone.utc`.

A7-L2 exercise 3

Hints. (1) What does Spark infer from a column that holds only None? (2) Where can you tell Spark the types instead? (3) Build a StructType and pass it to createDataFrame with a tuple row.

Answer. The bug is on line 3.

Fixed version:

```
1 def run_record(spark: SparkSession, run_id: str, rows: int, status: str,
    error_message: str | None) -> DataFrame:
2     """One run log row."""
3     schema = T.StructType([
4         T.StructField("run_id", T.StringType(), False),
5         T.StructField("rows_processed", T.LongType(), False),
6         T.StructField("status", T.StringType(), False),
```

```
        T.StructField("error_message", T.StringType(), True),
    ])
    return spark.createDataFrame([(run_id, rows, status, error_message)],
    schema)
```

Without a schema, the successful case has `error_message=None` only, and inference fails. A `StructType` declares the type, so None is simply NULL.

Common wrong answers.

- Line 7: Passing None is correct for a success. The problem is how Spark decides the column's type.

A7-L2 exercise 4

Hints. (1) Declare seven StructFields in order, with error_message nullable. (2) Build the row as a tuple in the same order. (3) `spark.createDataFrame([row], RUN_LOG_SCHEMA)`.

Answer.

```
from datetime import datetime, timezone

from pyspark.sql import DataFrame, SparkSession
from pyspark.sql import types as T

RUN_LOG_SCHEMA = T.StructType([
    T.StructField("run_id", T.StringType(), False),
    T.StructField("pipeline_name", T.StringType(), False),
    T.StructField("rows_processed", T.LongType(), False),
    T.StructField("status", T.StringType(), False),
    T.StructField("error_message", T.StringType(), True),
    T.StructField("started_at", T.TimestampType(), False),
    T.StructField("finished_at", T.TimestampType(), False),
])


def run_record(spark: SparkSession, run_id: str, pipeline_name: str,
    rows_processed: int, status: str, started_at: datetime, error_message:
    str | None = None) -> DataFrame:
    """One run log row typed by RUN_LOG_SCHEMA.

    Args:
        spark: Active session.
        run_id: Unique id of the run.
        pipeline_name: Job name.
        rows_processed: Rows written by the run.
        status: SUCCESS or FAILED.
        started_at: UTC start time.
        error_message: Error text for a failed run.

    Returns:
        A one-row DataFrame ready to append to the run log table.
```

```
    """
    finished_at = datetime.now(timezone.utc)
    row = (run_id, pipeline_name, rows_processed, status, error_message,
started_at, finished_at)
    return spark.createDataFrame([row], RUN_LOG_SCHEMA)
```

The module-level schema makes every record the same shape, whatever its values. Tuples in schema order are the simplest row format.

A7-L2 exercise 5

Hints. (1) The id must exist before any work is logged. (2) try, except, finally is Python's fixed order. (3) finally runs whether the work succeeded or raised.

Answer.

1. Create run_id and started_at
2. try: run the layer and read rows_processed from the commit metrics
3. Set status SUCCESS
4. except: set status FAILED and error_message, log.exception, then raise
5. finally: append the run record to the run log table

`finally` runs after both paths, so every run is recorded with its true status, and the re-raise in `except` still fails the job.

Common wrong answers.

- The except block belongs before finally in the try statement, and the record must see the FAILED status the except block sets.
- Create the run id first, so log lines during the work carry it.

A7-L3 Alert on failures, and on the silence that hides them

A7-L3 exercise 1

Hints. (1) Does the run fail? (2) When does on_failure send? (3) No failure, no email.

Answer. b) Nothing. Because: r1) The task returns normally, so the run succeeds and on_failure never fires.

Catching and not re-raising turns a failure into a success, so the failure notification never fires. Log, record FAILED, then re-raise.

Common wrong answers.

- `a`: on_failure fires when the run fails. Swallowing the exception made the run succeed.

- `c`: There is no warning notification for a caught exception. The run simply succeeds.
- `d`: Only if on_success were configured. It is not.
- `b|r2`: Right outcome, wrong reason. Notifications work for every task type. The run succeeded because the exception was swallowed.

A7-L3 exercise 2

Hints. (1) The event key names what happened to the run. (2) Send to the on-call group address. (3) `on_failure` and `data-oncall@example.org`.

Answer.

```
email_notifications:
  on_failure:
    - data-oncall@example.org
```

`email_notifications.on_failure` lists the addresses to notify when a run fails. Use a group address so the alert survives holidays and team changes.

Common wrong answers.

- g1 = `on_error`: The key is `on_failure`. Other keys include on_start and on_success.
- g2 = `sam.jones@example.org`: Send alerts to a group address. One person's inbox goes quiet when they are away.

A7-L3 exercise 3

Hints. (1) Which values does the table hold? (2) Does 'FAIL' equal 'FAILED'? (3) Use status = 'FAILED'.

Answer. The bug is on line 3.

Fixed version:

```
SELECT pipeline_name, run_id, error_message
FROM research_prod.research_governance.pipeline_run_log
WHERE status = 'FAILED'
  AND finished_at >= current_timestamp() - INTERVAL 1 DAY;
```

String comparison is exact, so `'FAIL'` never matches `'FAILED'`. Typed statuses in the writer (Literal) and a test query against real rows catch this.

Common wrong answers.

- Line 4: The one-day window is fine. Compare the status literal with what the table stores.

A7-L3 exercise 4

Hints. (1) Track every pipeline name, starting with no success. (2) Keep the latest SUCCESS finished_at per pipeline. (3) Stale when there is no success, or the latest is before now - max_age_hours.

Answer.

```python
from datetime import datetime, timedelta


def stale_pipelines(runs: list[dict[str, object]], now: datetime,
    max_age_hours: int = 26) -> list[str]:

    """Pipelines whose latest SUCCESS is older than max_age_hours, or that
    never succeeded.

     Args:
         runs: Run log rows with pipeline_name, status and finished_at.
         now: Current time, timezone-aware.
         max_age_hours: Largest acceptable age of the latest success.

     Returns:
         Sorted pipeline names.
     """
     cutoff = now - timedelta(hours=max_age_hours)
     latest: dict[str, datetime | None] = {}
     for r in runs:
         name = str(r["pipeline_name"])
         latest.setdefault(name, None)
         finished = r["finished_at"]
         if r["status"] == "SUCCESS" and isinstance(finished, datetime):
             current = latest[name]
             if current is None or finished > current:
                 latest[name] = finished
     return sorted(name for name, last in latest.items() if last is None
    or last < cutoff)
```

Recording every pipeline first means one that never succeeded still appears. Comparing only the latest success stops a single later failure from raising a false alarm.

A7-L3 exercise 5

Hints. (1) Which problems make the job fail? (2) Which problems leave the job successful, or never started? (3) Is every change worth waking someone?

Answer.

- Exceptions fail the job: the failure notification handles them, provided nothing swallows them.

- A job that never runs cannot fail: detect the missing SUCCESS with a query.
- A success that did nothing: query for rows_processed = 0.
- Normal variation is not an alert. Alerts that fire on noise get ignored.

Failures that raise are covered by job notifications. Silence and empty successes need scheduled queries. Noise needs no alert at all.

Common wrong answers.

- paused = `job`: A paused job never runs, so it never fails, and the failure notification never fires.
- zero_rows = `job`: The job succeeds, so no failure notification fires. Only a query on the run log sees the zero.
- slow = `query`: A one-minute change is noise. Alerting on it trains people to ignore alerts.